

Section 4: Architectural Patterns & Styles

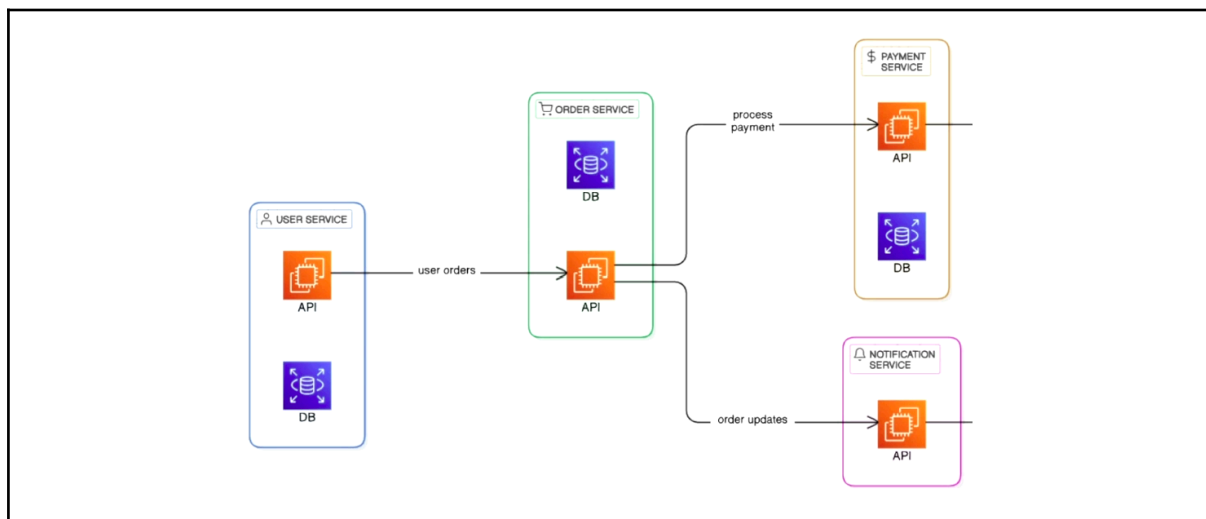
Microservices Architecture

In this session, we will explore how modern applications are designed using microservices architecture, how they differ from monolithic systems and why leading tech companies like Netflix, Uber and Amazon have adopted this approach.

By the end of this session, you will understand the key components of microservices, how services, how services communicate, the common challenges of microservices and deployment strategies to build scalable and resilient systems.

What is Microservices Architecture?

Microservices architecture is a software design pattern that structures an application as a collection of small, independent services, each handling a specific business function. Unlike monolithic applications, where all components are tightly integrated, microservices are loosely coupled, meaning each service operates independently and communicates via APIs.

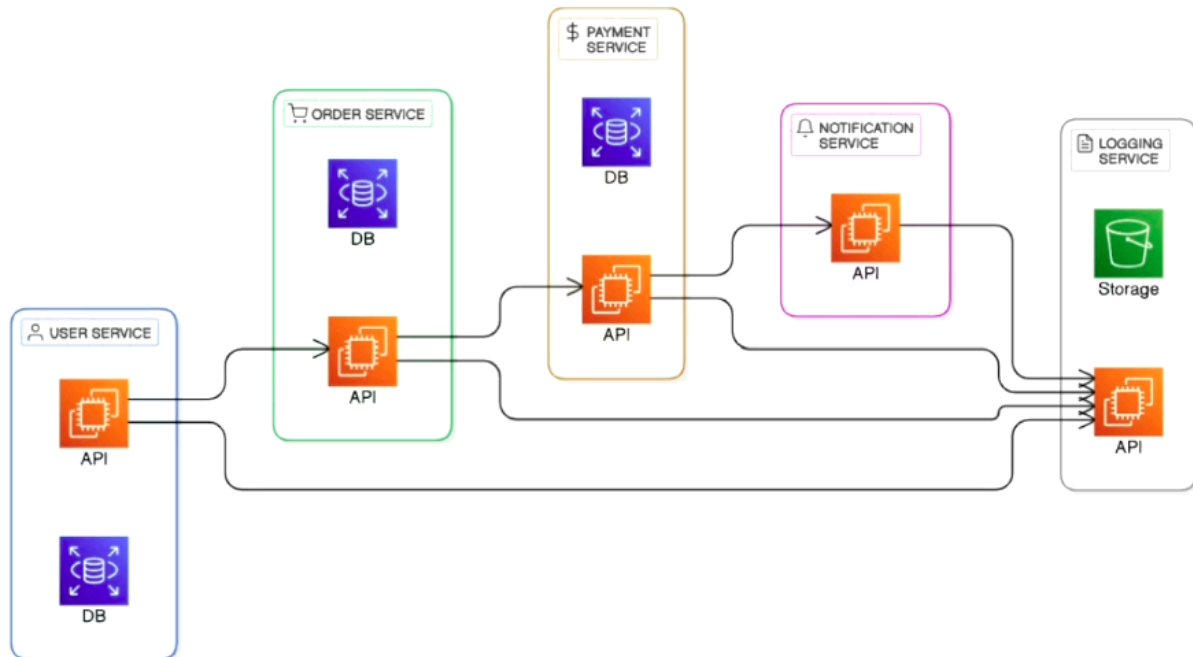


The key characteristics of microservices are:

1. They are independently deployable. Each service can be deployed, updated and maintained without affecting any other service.
2. They are loosely coupled and modular. Services interact with each other through a well-defined API, but by themselves, they remain self-contained.
3. They are highly scalable. You can scale individual services based on demand and failure in any one of the services doesn't bring down the whole system.

This approach enables organizations to build applications that are more resilient, scalable and maintainable, making microservices a preferred choice for modern software development.

Identifying and Structuring Microservices



One of the key challenges in microservices architecture is deciding how to break down an application into several individual services. How to identify a microservice?

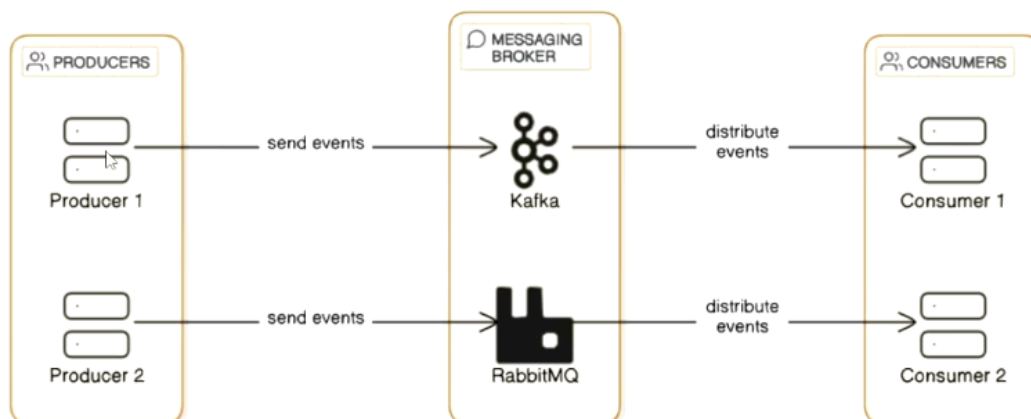
- **Business Capabilities:** Each microservice should align with a specific business function (e.g., payment, order or user). This ensures clear ownership and logical separation.
- **Single Responsibility Principle:** A microservice should do one thing well. If the service handles multiple unrelated concerns, it may need further decomposition.
- **Data Ownership:** Each microservice should manage its own database. Shared databases create dependencies and reduce the benefits of independent deployments.
- **Independently Deployable:** Microservices should be independently deployable, meaning updates to one service should not require redeploying the entire application, which includes other services.

Similarly, how can we structure microservices?

- **Decomposed by Business Domain:** We can use the domain driven design to break the system into logical domains. This helps us in grouping services based on business needs.
- **Define Clear APIs:** We ensure that microservices communicate through a well-defined API, whether via REST API, gRPC or event driven messaging.
- **Choose the Right Granularity:** Too large and we end with a monolith again. Too small and we will increase the complexity with excessive service interactions. So we need to size our microservice with a very careful thought.
- **Implement Observability:** Logging, monitoring and tracing help in tracking interactions between services and identifying the issues early.

By following these principles, we can design a scalable and maintainable microservices architecture.

Communication in Microservices



In microservices architecture, services need to communicate with each other to exchange data and process requests. The two main communication models are synchronous and asynchronous.

- **Synchronous Communication:** In synchronous communication, services wait for a response before proceeding. There are 2 common methods for synchronous communication:
 - **REST APIs:** REST is the most commonly used approach as it follows HTTP standards and is easy to implement. However, since it relies on text-based communication, either JSON or XML, it can introduce higher latency.
 - **gRPC:** It is the alternative to REST, which is a high performance communication protocol that uses a binary format called protocol buffers. It is much more efficient than REST and is ideal for microservices which require low latency and high throughput.

- **Asynchronous Communication:** In many cases, microservices don't need to wait for the immediate response. This is where asynchronous communication comes in using event-driven messaging. Event-driven architecture relies on message brokers like Kafka or RabbitMQ. These systems enable microservices to publish and subscribe to events, reducing dependencies between services improving scalability.

For example, when a user places an order in an e-commerce system, the order service can publish an event like order created. The other services like payment service and inventory service can react to this event, without any direct dependencies.

So choosing the right communication strategy will depend on your use case. REST and gRPC, meaning synchronous communication, is great when services need immediate responses. On the contrary, event-driven asynchronous communication is ideal when services need loose coupling and scalability. By leveraging the right mix of synchronous and asynchronous communication, we can build a very highly efficient and scalable microservices system.

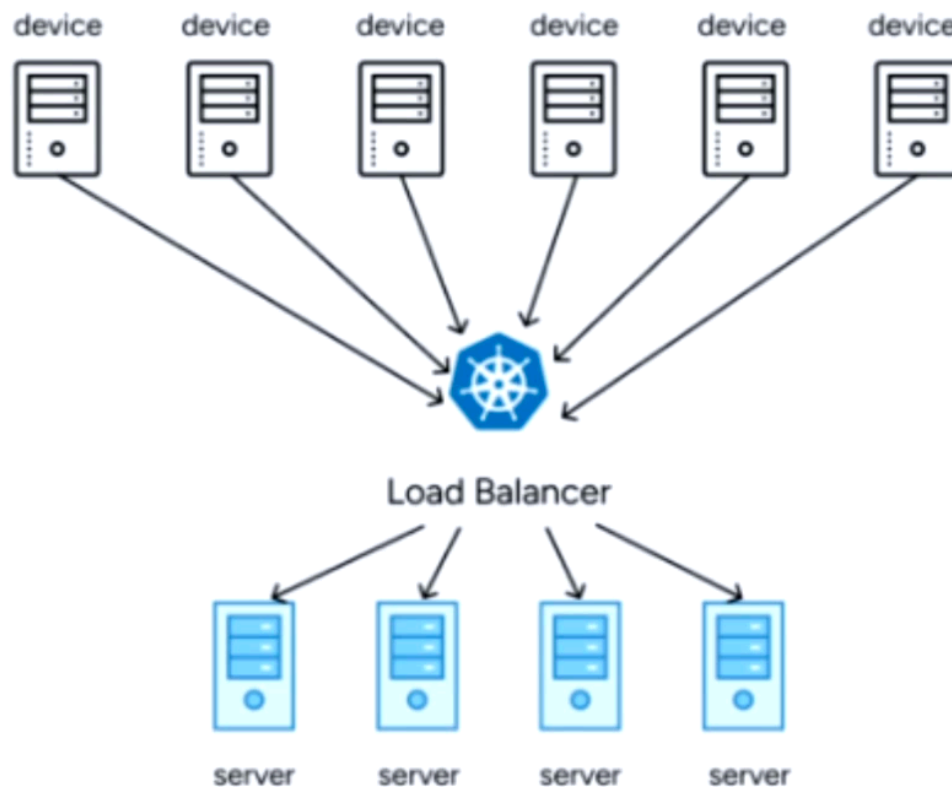
Challenges of Microservices

It's not always positive about microservices. While it does bring scalability, flexibility and independent deployment, they also introduce several challenges that the team needs to address. The below are some of the challenges faced by teams:

- ➔ **Data Consistency:** Unlike monolithic applications where a single database ensures strong consistency, microservices often use distributed databases. This leads to the concept of eventual consistency where the changes take time to propagate across services. For example, in an e-commerce system, if an order service updates an order status, it may take a moment for the inventory service to reflect the change. Event driven messaging can help in handling these delays.
- ➔ **Distributed Tracing:** Since microservices interact over a network, tracking a single request across multiple services is challenging. Debugging issues like failures, delays, and incorrect responses require distributed tracing.
- ➔ **Network Overheads:** Microservices rely heavily on API calls for communication, introducing network latency. More services mean more inter-service calls, which can be slow and bring down performance. Optimization's like using gRPC's binary format, caching, and aggregating requests can help in reducing these overheads.
- ➔ **Security Concerns:** In a microservices environment, every service needs proper authentication, authorization, and data protection. Unlike monolithic applications where the security is centralized, microservices must implement security at multiple levels for secure communication between these services.

Addressing all these challenges requires the right design patterns, tools, and best practices, ensuring that microservices remain scalable, secure and reliable. But we need to acknowledge the fact that implementing microservices brings additional levels of complexity, and we need to be ready to handle this complexity in our application.

Scaling Strategies in Microservices



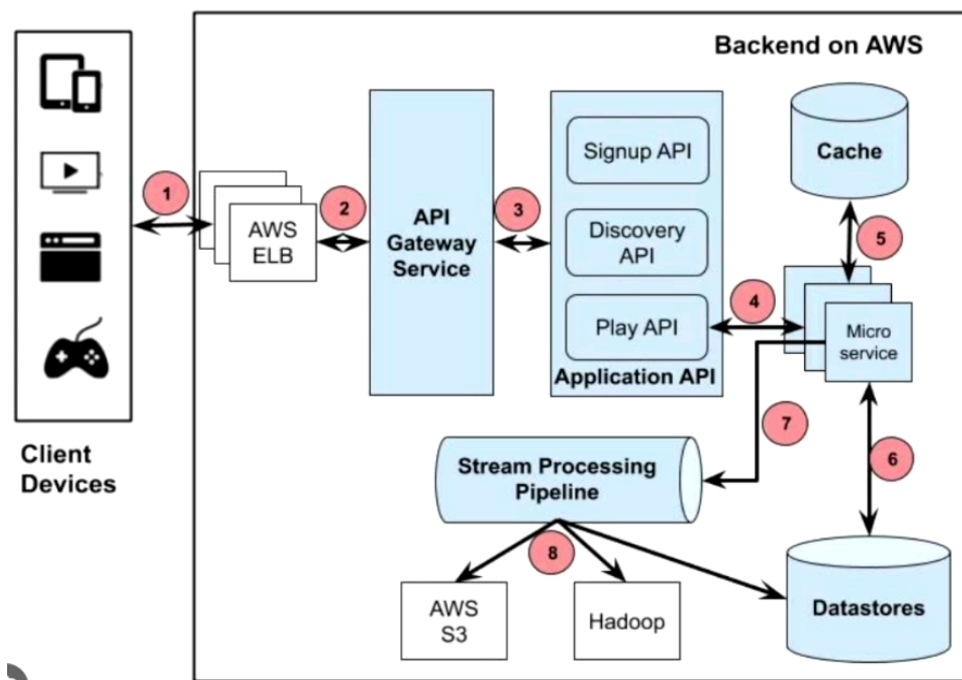
Scaling is one of the biggest advantages of microservices. Unlike monolithic applications where scaling requires increasing resources for a single application, microservices allow us to scale individual components independently. The below are some of the key strategies used for scaling in microservice-based applications:

- **Horizontal Scaling:** Instead of making a single server more powerful, which was called the vertical scaling, we add multiple instances of a microservice and distribute traffic among them. This ensures high availability and load balancing. For example if the order service in an ecommerce system is under heavy load, we can deploy additional instances of the order service to handle more requests.
- **Auto Scaling:** Manually adding or removing instances is insufficient. So we use auto scaling solutions from the cloud vendors or like kubernetes horizontal pods, and they will help us in automatically scaling the service whenever there is a high load on that service. These tools automatically scale these services up or down based on CPU memory and request load.

- **Sharding & Database Scaling:** As traffic increases, database performance can become a bottleneck. One solution is sharding where we split the database into smaller partitions based on a unique key. Another approach is read replicas, where we have multiple copies to handle read heavy traffic efficiently.

So by using the combination of horizontal scaling, auto scaling and database optimization, we can ensure that microservices handle increasing workloads efficiently without downtime.

Real-World Examples of Microservices



Let's look at some of the real world most successful companies that have adopted microservices to scale and improve their performance.

- **Netflix:** Netflix relies heavily on microservices to handle video streaming, personalization and recommendation. They have decoupled each function into separate services and they can scale different components independently and ensure a smooth experience for millions and millions of users.
- **Uber:** Uber also uses microservices to scale ride matching payment and navigation independently.
- **Amazon:** Amazon has a microservices based on architecture where each function such as search, payment, recommendation runs in its own service. This modularity ensures that Amazon can handle massive traffic and make real time changes without downtime.

So these companies are great examples of how microservices enable scalability, flexibility and resilience, making them able to meet the needs of millions and millions of customers everyday.

Summary & What's Next

- Microservices = Scalability, Fault Tolerance, Faster Development
- Requires API Gateway, Service Discovery, Load Balancing
- Challenges: Data consistency, debugging, deployment complexity
- Real-world usage: Netflix, Uber, Amazon
- Next Lecture:
 - Event-Driven Architectures!

Interview Questions on Microservices

- What are microservices, and how do they differ from monolithic architecture?
- What are the benefits and challenges of microservices?
- Explain the role of an API Gateway in a microservices system.
- How do microservices communicate with each other?
- How can you ensure data consistency in a microservices-based system?